NETSENTINEL

Network Traffic Analyzer and Intrusion Detection System

A Web-Based Platform for Real-Time Network Monitoring and Threat Detection

MINI PROJECT REPORT

Submitted by

SAMEER SHA

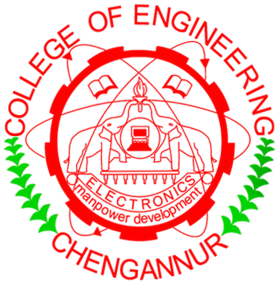
CHN25MCA-2045

to

APJ Abdul Kalam Technological University

in partial fulfillment of the requirements for the award of Degree in

Master of Computer Applications



DEPARTMENT OF COMPUTER ENGINEERING

COLLEGE OF ENGINEERING CHENGANNUR, ALAPPUZHA

OCTOBER 2026

DEPARTMENT OF COMPUTER ENGINEERING

COLLEGE OF ENGINEERING CHENGANNUR

ALAPPUZHA

CERTIFICATE

This is to certify that the project report titled ***NETSENTINEL: NETWORK TRAFFIC ANALYZER AND INTRUSION DETECTION SYSTEM*** *is a bonafide record of the* ***20MCA245 MINI PROJECT*** *presented by* ***SAMEER SHA*** *(CHN25MCA-2045), Third Semester Master of Computer Applications student, under our guidance and supervision. This project is submitted in partial fulfillment of the requirements for the award of the degree* ***Master of Computer Applications*** *of* ***APJ Abdul Kalam Technological University****.*

Smt. Beena K
Project Guide
Assistant Professor
Dept. of Computer Engineering

Smt. Josmi Jose
Project Coordinator
Assistant Professor
Dept. of Computer Engineering

Dr. Renu George
Professor
Head of the Department
Dept. of Computer Engineering

DECLARATION

I undersigned hereby declare that the project report **NETSENTINEL: NETWORK TRAFFIC ANALYZER AND INTRUSION DETECTION SYSTEM**", submitted for partial fulfillment of the requirements for the award of degree of Master of Computer Application of the APJ Abdul Kalam Technological University, Kerala is a bonafide work done by me under the supervision of **Smt. Beena K**, Assistant Professor, Department of Computer Engineering. This submission represents my ideas in my own words, and where ideas or words of others have been included, I have adequately and accurately cited and referenced the original sources. I also declare that I have adhered to the ethics of academic honesty and integrity and have not misrepresented or fabricated any data, idea, fact, or source in this submission. I understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not been previously formed the basis for the award of any degree, diploma, or similar title of any other University.

Place: Chengannur **SAMEER SHA**

Date : **CHN25MCA-2045**

ACKNOWLEDGEMENT

This work would not have been possible without the support, guidance, and cooperation of many individuals. First and foremost, I offer my humble gratitude to Almighty God, whose grace, blessings, and strength enabled me to undertake and successfully complete this project work.

I would like to express my sincere gratitude to **Dr. Hari V.S**, The Principal, College of Engineering Chengannur, for providing the necessary facilities, infrastructure, and academic atmosphere conducive to project development. I extend my deep appreciation to **Dr. Renu George**, Professor and Head of the Department of Computer Engineering, for her continuous support, administrative facilitation, and encouragement throughout the semester.

I am profoundly grateful to my project guide, **Smt. Beena K**, Assistant Professor, Department of Computer Engineering, for her invaluable guidance, technical suggestions, constructive critiques, and constant encouragement at every stage of the project. I also extend my sincere thanks to our project coordinators, **Smt. Josmi Jose**, **Smt. Santhy Viswam**, and **Smt. Celin T**, Assistant Professors, Department of Computer Engineering, for their meticulous planning, periodic evaluations, and timely guidance.

Finally, I express my heartfelt gratitude to all faculty members of the Department of Computer Engineering, my dear classmates, and my family members for their unwavering moral support, valuable feedback, and encouragement throughout this endeavour.

SAMEER SHA

CHN25MCA-2045

ABSTRACT

Network security monitoring in contemporary digital environments is frequently impaired by the high volume of raw network telemetry, the operational complexity of legacy packet inspection utilities, and the lack of accessible, unified visualization platforms. Traditional command-line packet sniffers and standalone intrusion detection engines require specialized expertise to configure, lack integrated multi-host tracking, and fail to provide immediate, actionable situational awareness for security analysts. To address these operational challenges, this project presents NetSentinel, a comprehensive web-based Network Traffic Analyzer and Intrusion Detection System engineered to deliver real-time packet inspection, automated behavioral threat detection, and centralized security operations management. NetSentinel incorporates a decoupled, multi-tier architecture comprising a high-performance Python packet capture and intrusion detection engine, a robust Node.js and Express RESTful application programming interface with Socket.io WebSocket event streaming, an enterprise MySQL relational database, and an interactive React web dashboard designed with a soft claymorphic aesthetic. The Python engine captures live Layer 3 and Layer 4 packets using Scapy or generates reproducible multi-host traffic through an integrated synthetic simulator, evaluating telemetry against configurable sliding-window heuristic detection rules for SYN floods, port scans, ICMP floods, suspicious DNS activity, malicious payloads, and malformed headers. Detected anomalies trigger immediate database alerts with a thirty-second cooldown cache and stream instantaneously to connected clients. The platform enforces strict role-based access control between security analysts and administrators, facilitating real-time throughput charting, protocol distribution breakdown, automated host discovery with threat scoring, alert triage, rule tuning, and on-demand generation of executive PDF and raw CSV forensic reports. By integrating wire-speed packet ingestion, explainable signature detection, and responsive telemetry visualization into a cohesive web platform, NetSentinel significantly improves network observability, threat triage efficiency, and security operations management.

CONTENTS

Declaration i

Acknowledgement ii

Abstract iii

List of Figures vii

List of Tables viii

1 INTRODUCTION 1

1.1 Project Area 1

1.2 Objectives 2

2 LITERATURE REVIEW 4

2.1 Network Traffic Analyzer: Real-Time Detection and Malicious Activity Identification 4

2.2 Anomaly Detection Module for Network Traffic Monitoring in Public Institutions . 4

2.3 Network Intrusion Detection System Using Anomaly Detection Techniques 5

2.4 Exploring Traffic Patterns Through Network Programmability: Introducing SDNFlow 5

2.5 Deep-IDS: A Real-Time Intrusion Detector for IoT Nodes Using Deep Learning . . 6

2.6 Gap Identified 6

3 PROBLEM DEFINITION AND MOTIVATIONS 7

3.1 Existing System 7

3.2 Limitations 7

3.3 Problem Statement 8

3.4 Proposed System 8

4 REQUIREMENT ANALYSIS 10

4.1 Hardware Requirements 10

4.2 Software Requirements 10

4.3 System Features 12

4.4 Non-Functional Requirements 13

4.4.1 Performance Requirements . . . 13
4.4.2 Quality Requirements . . . 13

5 DESIGN AND IMPLEMENTATION 15

5.1 Overall Design . . . 15
5.2 System Design . . . 15
5.2.1 Client-Side Design . . . 15
5.2.2 Server-Side Design . . . 15
5.3 System Architecture . . . 16
5.4 Use Case Diagram . . . 17
5.5 Data Flow Diagrams (DFD) . . . 18
5.6 Database Design . . . 21
5.6.1 Database Requirements . . . 21
5.6.2 Database Schema . . . 21
5.7 Threat Detection and Packet Ingestion Lifecycle . . . 22
5.8 Module Description . . . 23
5.8.1 Packet Capture and Threat Engine Module . . . 23
5.8.2 Backend API and Telemetry Processing Module . . . 24
5.8.3 Frontend Security Operations Dashboard Module . . . 25

6 REPORT OF PROJECT IMPLEMENTATION 26

6.1 Implementation Plan . . . 26
6.1.1 Setup and Environment Preparation . . . 26
6.1.2 Backend Implementation . . . 26
6.1.3 Frontend Implementation . . . 27
6.1.4 Security and Data Handling . . . 27
6.1.5 Project Schedule . . . 28
6.2 Testing and Various Types of Testing Used . . . 28
6.2.1 Unit Testing . . . 29
6.2.2 Integration Testing . . . 31
6.2.3 System Testing . . . 33

7 RESULTS AND DISCUSSION 35

7.1 Advantages . . . 35

7.2 Limitations 36
7.3 Screenshots 37

8 CONCLUSION AND FUTURE SCOPE **40**
8.1 Conclusion 40
8.2 Future Scope 40

REFERENCES **42**

LIST OF FIGURES

5.1 System Architecture 16
5.2 Use Case Diagram 17
5.3 Data Flow Diagram - Level 0 (Context Level) 18
5.4 Data Flow Diagram - Level 1 (User / Analyst Flow) 19
5.5 Data Flow Diagram - Level 1 (Administrator Flow) 20
5.6 Relational Database Schema Diagram 21

7.1 Standard User Dashboard with Real-Time Telemetry 37
7.2 Administrator Operations Dashboard with Capture Controls 37
7.3 Live Traffic Explorer with Protocol Filters 38
7.4 Incident Investigation Drawer with Hex Payload Preview 38
7.5 Security Alerts Management and Triage View 39
7.6 Discovered Network Devices Inventory with Risk Scores 39
7.7 Detection Rules Configuration and Sensitivity Tuning 39

LIST OF TABLES

5.1 Database Entities and Key Attributes 22

6.1 Project Implementation Schedule and Milestones 28
6.2 Unit Testing Test Cases 30
6.3 Integration Testing Test Cases 32
6.4 System Testing Test Cases 34

CHAPTER 1

INTRODUCTION

1.1 Project Area

Modern digital communication networks serve as the backbone for organizational operations, enterprise resource access, cloud computing, and industrial automation. As institutions and corporate enterprises connect heterogeneous computing nodes across local area networks and distributed internet gateways, the volume, velocity, and architectural heterogeneity of network packets increase exponentially. Securing these environments requires continuous visibility into communications occurring across Layer 3 (Network) and Layer 4 (Transport) protocols. In the absence of structured, automated surveillance, malicious cyber operations such as distributed denial-of-service volumetric floods, reconnaissance scanning probes, command-and-control communication tunneling, and application-layer exploit attempts can traverse internal network segments without triggering timely defensive responses.

Conventional network security practices frequently depend on decentralized, isolated monitoring utilities. Command-line diagnostic tools such as tcpdump, specialized packet capture utilities like Wireshark, and heavy enterprise Intrusion Detection Systems (IDS) often impose steep operational barriers. Command-line tools require security personnel to manually formulate complex packet capture filters and parse high-density hexadecimal packet dumps, which impedes rapid situational assessment during ongoing cyber incidents. Conversely, enterprise commercial monitoring suites demand significant computational overhead, complex cluster deployments, and proprietary agent installations on every client host. Furthermore, traditional utilities frequently lack integrated web-based dashboards capable of translating raw packet captures into intuitive, real-time graphical representations of throughput, bandwidth utilization, host inventory, and incident severity.

‘**NetSentinel**” is designed and implemented as a full-stack, real-time Network Traffic Analyzer and Intrusion Detection System that unifies live packet interception, rule-based behavioral threat analysis, relational telemetry persistence, and claymorphic web visualization. Developed with a decoupled architecture, NetSentinel employs a Python-based packet capture service using Scapy alongside an enterprise Node.js Express RESTful backend, a MySQL 8.4 database, and a React web dashboard. The system incorporates dual operational modes—a live interface sniffer for physical

and wireless network cards and a synthetic multi-host simulator for controlled security evaluations—while enforcing strict role-based access control between Standard Users (Security Analysts) and Administrators. Designed for rapid deployment in local network environments, NetSentinel provides extensible sliding-window detection rules, automated host discovery with behavioral risk scoring, and one-click PDF and CSV forensic export capabilities.

1.2 Objectives

1. **To develop a centralized real-time packet capture and ingestion pipeline:** The system aims to capture live Ethernet, Wi-Fi, and simulated multi-host IP packets continuously, extracting protocol headers, IP addresses, port numbers, packet lengths, and TCP control flags for real-time security analysis.
2. **To engineer a multi-rule sliding-window intrusion detection engine:** The system aims to implement stateful and heuristic detection algorithms capable of identifying volumetric SYN floods, reconnaissance port scans, ICMP echo floods, suspicious DNS queries, signature-based payload attacks, and malformed network packets.
3. **To implement automated network device discovery and risk scoring:** The system aims to inspect observed traffic dynamically to identify active network hosts, resolve IP and MAC address pairs, classify hardware types, and compute empirical behavioral risk scores based on detected incident history.
4. **To establish an asynchronous event streaming and telemetry architecture:** The system aims to stream live throughput metrics, bandwidth statistics, protocol distributions, and threat alerts from the backend ingestion controllers to connected browser sessions using WebSocket communication.
5. **To provide strict role-based access control and administrative governance:** The system aims to enforce distinct privilege levels between read-only Security Analysts and SecOps Administrators, restricting capture engine execution, rule sensitivity tuning, alert triage, and user management to verified administrative users.
6. **To deliver an interactive, responsive claymorphic web dashboard:** The system aims to present comprehensive network intelligence through responsive Chart.js line and donut visualizations, real-time packet tables, and an interactive threat investigation drawer displaying hexadecimal payload previews.
7. **To generate standardized executive PDF and raw forensic CSV reports:** The system aims

to compile structured audit documentation and raw forensic packet archives on demand across configurable historical time windows to support compliance audits and forensic analysis.

8. **To provide a dual-mode execution environment with synthetic attack simulation:** The system aims to support both elevated physical packet sniffing via Npcap and an integrated synthetic multi-host traffic simulator that enables offline security testing, education, and benchmark verification without external hardware dependencies.

CHAPTER 2

LITERATURE REVIEW

2.1 Network Traffic Analyzer: Real-Time Detection and Malicious Activity Identification

This research paper, authored by A. Tamrakar, J. Chaturvedi, M. Poddar, and S. Kumari, investigates automated techniques for real-time packet analysis and anomalous traffic identification in enterprise networks. The authors present an architectural framework that intercepts incoming and outgoing data frames at the data-link layer, parsing network layer headers to reconstruct communication flows and flag volumetric irregularities. Their experimental findings emphasize that manual log auditing creates an unacceptable latency window during active denial-of-service and brute-force scanning events, whereas script-based real-time analysis provides rapid containment triggers. However, the study focuses primarily on terminal-based log alerts and lacks an integrated multi-tier database repository or a unified web dashboard for role-differentiated team collaboration. This work directly informs NetSentinel's core design by validating packet-level inspection pipelines; NetSentinel extends their concepts by integrating a MySQL relational database, an interactive React interface, and multi-rule sliding windows. [1]

2.2 Anomaly Detection Module for Network Traffic Monitoring in Public Institutions

In this study published in the journal Sensors, Ł. Wawrowski, A. Białas, A. Kajzer, and their co-authors propose an anomaly detection module developed within the RegSOC framework, specifically designed to secure public administrative networks and small-to-medium enterprise infrastructures. The authors evaluated multiple classification algorithms to optimize detection accuracy while establishing an accelerated offline model tuning routine. Their findings demonstrated that combining statistical baseline models with targeted heuristic classifiers yielded up to 100% balanced accuracy in isolating localized reconnaissance probes and irregular outbound bandwidth surges. Despite its high analytical precision, the RegSOC module is tightly coupled with enterprise Security Operations Center (SOC) hardware and requires extensive computational resources for feature transformation.

This research is highly relevant to NetSentinel as it demonstrates the efficacy of heuristic thresholds for localized traffic monitoring. NetSentinel differentiates itself by providing a lightweight, low-overhead Node.js and Python architecture that executes efficiently on standard workstation hardware. [2]

2.3 Network Intrusion Detection System Using Anomaly Detection Techniques

Presented at the IEEE 20th International Conference on Intelligent Computer Communication and Processing, this investigation by D. Oroian, R. Bolboaca, A.-S. Roman, and V. Dobrota evaluates neural network classifiers for real-time intrusion detection against standard cyber threat benchmark corpora including KDDCUP99 and UNSW-NB15. Utilizing the Cisco TRex synthetic traffic generator along with tcpdump and Zeek ingestion pipelines, the authors benchmarked Isolation Forests, Multi-Layer Perceptrons, and Autoencoder architectures. Their experimental evaluation confirmed that lightweight binary Multi-Layer Perceptrons serve as a highly effective frontline filter for modern network traffic anomalies. Nevertheless, the study highlighted that deep learning models introduce non-negligible computational latency and operate as opaque black boxes, obscuring the precise protocol flags responsible for triggering an intrusion alarm. This paper strongly supports NetSentinel's emphasis on explainability; NetSentinel implements transparent, rule-based sliding-window algorithms where every alert explicitly exposes its triggering threshold, protocol attributes, and payload signatures. [3]

2.4 Exploring Traffic Patterns Through Network Programmability: Introducing SDNFlow

Published in IEEE Access, this paper by J. Buzzio-García, J. Vergara, S. Ríos-Guiral, and colleagues introduces the SDNFlow dataset and examines automated traffic classification in programmable OpenFlow network fabrics. The authors collected flow-level statistics across simulated software-defined networking switches, evaluating classic machine learning models such as K-Nearest Neighbors, Random Forests, and Support Vector Machines to distinguish legitimate transmissions from distributed denial-of-service floods and port scans. Their analysis demonstrated that statistical metrics—specifically flow duration, byte-to-packet ratios, and destination port diversity—provide robust discriminators for identifying reconnaissance probes with greater than 98% precision. However, their architecture relies entirely on centralized SDN OpenFlow controllers, limiting its direct applicability to conventional non-SDN local area network topologies. NetSentinel bridges this gap by calculating

analogous sliding-window metrics directly from raw physical socket streams or synthetic simulator engines, bringing flow-level discriminative insights to standard campus and office networks. [4]

2.5 Deep-IDS: A Real-Time Intrusion Detector for IoT Nodes Using Deep Learning

In this research published in IEEE Access, S. Racherla and colleagues develop an edge-oriented intrusion detector tailored for interconnected Internet of Things (IoT) ecosystems characterized by constrained memory and processing capabilities. The authors designed a real-time feature extraction pipeline that processes incoming telemetry streams to identify malicious probing and volumetric floods before they compromise downstream network appliances. Their findings highlight that edge monitoring systems must maintain an ultra-low memory footprint to prevent packet dropping during peak throughput bursts. NetSentinel draws inspiration from this resource-conscious paradigm, incorporating decoupled asynchronous batch processing between the Scapy packet capture worker and the Express ingestion API, ensuring zero dropped frames under standard local network monitoring loads. [5]

2.6 Gap Identified

A critical synthesis of contemporary literature reveals significant architectural and operational gaps in existing network monitoring solutions. While academic literature has explored sophisticated neural network classifiers, such models require heavy computational infrastructure, extensive historical training datasets, and parameter re-tuning, while offering limited transparency into why a given packet was classified as anomalous. On the other hand, traditional signature-based utilities such as Snort or Zeek generate voluminous unstructured text logs that require dedicated log management pipelines (e.g., Elasticsearch, Logstash, Kibana) to become usable by human analysts. Furthermore, existing open-source monitoring utilities rarely incorporate integrated, end-to-end device discovery with behavioral risk scoring, role-based administrative control over capture engines, and built-in forensic PDF/CSV report compilers in a unified, lightweight package. There is a decisive need for an accessible, self-contained, and explainable network traffic analyzer that pairs multi-rule sliding-window detection with a modern web dashboard. NetSentinel directly addresses these gaps by combining lightweight packet parsing, deterministic rule transparency, relational MySQL telemetry persistence, and an intuitive claymorphic interface into an integrated system.

CHAPTER 3

PROBLEM DEFINITION AND MOTIVATIONS

3.1 Existing System

In conventional computing environments, network traffic analysis and perimeter intrusion detection typically rely on disparate, disconnected software utilities or costly enterprise appliances. System administrators and network security personnel routinely utilize command-line network diagnostic tools such as tcpdump, Wireshark, or basic operating system socket monitors to inspect localized packets. For organizational intrusion detection, enterprises often deploy legacy signature-matching software or appliance firewalls that emit alert notifications via syslog protocols to remote terminal servers. When investigating a suspicious incident, security analysts are forced to capture raw packet files, transfer dump archives to isolated workstations, open massive PCAP logs in desktop analysis applications, and manually correlate IP addresses and timestamps against server error logs.

3.2 Limitations

- **Steep Technical Barrier and Lack of Unified Interfaces:** Conventional tools such as tcpdump require administrators to formulate intricate Berkeley Packet Filter (BPF) syntax from terminal consoles, lacking accessible web dashboards that display live operational context.
- **High Latency in Threat Triage and Remediation:** Manual extraction and retrospective examination of raw PCAP logs introduce substantial delays, allowing hostile reconnaissance probes and volumetric denial-of-service attacks to compromise services before detection.
- **Absence of Automated Device Discovery and Profiling:** Legacy packet sniffers display individual packet transactions without automatically resolving hardware MAC addresses, vendor types, or maintaining persistent behavioral host profiles within local subnets.
- **Opaque Alert Scoring and Explainability Gaps:** Commercial intrusion detection appliances often produce opaque severity rankings without clearly presenting the exact threshold values, sliding time windows, or packet payload signatures that triggered the alarm.
- **High Computational Overhead and Resource Consumption:** Enterprise security suites such as Splunk or Snort require dedicated multi-node server clusters, extensive disk arrays,

and high memory allocations, making them impractical for lightweight campus laboratories or small-to-medium offices.

- **Decoupled Packet Archiving and Database Storage:** Traditional systems separate structured event data from raw frame captures, requiring cumbersome manual correlation between relational incident databases and rotating binary PCAP storage files.
- **Lack of Role-Based Governance for Collaborative Teams:** Standalone diagnostic utilities operate under universal operating system permissions, lacking distinct functional segregation between read-only security analysts and privileged administrative operators.
- **Inflexible Testing Environments Without Hardware Coupling:** Most packet capture suites fail to operate without physical network interface cards configured in promiscuous mode with specialized drivers, preventing secure offline demonstration, simulation, and academic verification.
- **Cumbersome Forensic Reporting and Audit Workflows:** Exporting audit records from existing monitoring software involves executing custom SQL queries or terminal scripts, lacking one-click generation of professional executive PDF summaries and raw CSV forensic extracts.

3.3 Problem Statement

To design, develop, and evaluate a lightweight, real-time, web-based Network Traffic Analyzer and Intrusion Detection System capable of intercepting live and simulated network packets, performing automated multi-rule sliding-window threat detection across transport and application layer protocols, tracking discovered subnet devices with empirical risk scoring, and delivering immediate situational awareness, alert triage, and forensic reporting through an intuitive, role-governed web dashboard.

3.4 Proposed System

The proposed system, **NetSentinel**", is an end-to-end network security platform that bridges the divide between low-level packet capture engines and modern web visualization interfaces. NetSentinel is built upon a decoupled multi-tier architecture consisting of a Python packet capture and intrusion detection engine, a Node.js Express REST API server with Socket.io streaming, an enterprise MySQL 8.4 database, and a React web dashboard. The system continuously extracts protocol metadata, evaluates traffic flows against six configurable detection rules, maintains an automated inventory of discovered devices, and facilitates seamless alert management and audit reporting.

The key features of the proposed system include:

1. **Dual-Mode Packet Ingestion Pipeline:** Supports both live promiscuous hardware packet capture using Scapy across Ethernet and Wi-Fi adapters, and a high-fidelity synthetic traffic simulator that injects multi-host traffic and reproducible attack scenarios.
2. **Sliding-Window Rule-Based Threat Detection:** Implements stateful sliding-window algorithms and payload inspection to detect TCP SYN floods, port scans, ICMP floods, suspicious DNS tunneling, signature-based malicious payloads, and malformed headers.
3. **Real-Time WebSocket Telemetry Streaming:** Broadcasts live packet throughput metrics, protocol distribution breakdowns, bandwidth utilization, and threat alerts instantaneously to connected client dashboards via Socket.io.
4. **Automated Host Discovery and Behavioral Risk Scoring:** Dynamically identifies network hosts from packet streams, maps IP and MAC relationships, categorizes devices (routers, servers, workstations, IoT), and computes cumulative risk scores.
5. **Interactive Claymorphic Web Operations Dashboard:** Provides an aesthetically refined Soft Claymorphism user interface featuring dynamic Chart.js visualizations, live telemetry tables, and an interactive threat investigation drawer with hexadecimal payload previews.
6. **Strict Role-Based Access Control (RBAC):** Enforces strict permission boundaries separating read-only Security Analysts (user) from privileged SecOps Administrators (admin), safeguarding capture controls and rule sensitivity settings.
7. **Configurable Rule Sensitivity and Active Toggling:** Enables administrators to adjust detection threshold values, time windows, and severity classifications dynamically via the web interface without restarting the background detection engine.
8. **Dual Persistence with Relational Data and Rotating PCAP Archiving:** Stores structured telemetry, alerts, and device data in a normalized MySQL relational schema while concurrently maintaining rotating binary PCAP files on disk for deep forensic packet analysis.
9. **One-Click Forensic and Executive Audit Reporting:** Compiles executive PDF summaries complete with statistical charts and generates raw forensic CSV telemetry extracts across custom time ranges on demand.

CHAPTER 4

REQUIREMENT ANALYSIS

4.1 Hardware Requirements

- **Processor:** 64-bit Dual-Core x86-64 processor (Intel Core i3/i5 or AMD Ryzen 3 equivalent, 2.0 GHz or higher).
- **System Memory (RAM):** Minimum 4 GB RAM (8 GB recommended for simultaneous execution of MySQL server, Express API, Python capture engine, and React development server).
- **Storage Capacity:** Minimum 10 GB of available solid-state storage for database storage, system logs, and rotating PCAP capture files.
- **Network Adapter:** Integrated 10/100/1000 Mbps Gigabit Ethernet controller or 802.11ac/ax Wi-Fi network interface card supporting promiscuous socket mode for live sniffing.
- **Display Resolution:** Standard monitor supporting 1366 $\times$ 768 resolution minimum (1920 $\times$ 1080 Full HD recommended for full claymorphic dashboard visualization).

4.2 Software Requirements

- **Operating System:**
  - Microsoft Windows 10 / Windows 11 (64-bit) (primary development and verification platform).
  - Compatible with Ubuntu Linux 22.04 LTS / Debian 12 for containerized headless deployments.
- **Programming Languages and Runtimes:**
  - **Python v3.10+ / v3.14:** Core runtime for packet capture, Scapy protocol disassembly, and sliding-window intrusion detection algorithms.
  - **Node.js v18.0.0+ / v20.0.0+ / v24.0.0+:** JavaScript runtime hosting the Express REST API backend and Socket.io event engine.
  - **JavaScript (ECMAScript 2022) / JSX:** Client-side language powering the interactive React web dashboard.

- **Frameworks and Web Engines:**
  - **Express.js v4.19.2:** Minimalist web framework managing REST endpoints, authentication middleware, and route dispatching.
  - **React v18.3.1:** Component-based user interface framework utilizing virtual DOM reconciliation and state hooks.
  - **Vite v5.2.0:** Next-generation frontend build utility providing lightning-fast Hot Module Replacement (HMR).
- **Database Management System:**
  - **MySQL Server v8.0 / v8.4 Community Edition:** Relational database management system with InnoDB storage engine, foreign key enforcement, and utf8mb4 collation.
- **Core Libraries and Packages:**
  - **Scapy v2.8.0:** Python packet manipulation and promiscuous sniffing library for protocol decoding.
  - **Socket.io v4.7.5 & Socket.io-client v4.7.5:** Bidirectional, low-latency WebSocket communication engine.
  - **mysql2 v3.9.7:** High-performance MySQL driver supporting prepared statements and promise-based connection pooling.
  - **bcryptjs v2.4.3:** Cryptographic library for secure salted password hashing using 10 salt rounds.
  - **jsonwebtoken v9.0.2:** Stateless authentication token issuance and verification engine.
  - **pdfkit v0.15.0:** Server-side vector document compiler for executive audit report generation.
  - **csv-stringify v6.5.0:** High-throughput streaming CSV conversion library for raw forensic data exports.
  - **Chart.js v4.4.2 & react-chartjs-2 v5.2.0:** Responsive HTML5 canvas charting library for real-time telemetry graphs.
  - **Lucide React v0.378.0:** Vector icon library providing modern claymorphic navigation iconography.
- **Development Tools and Utilities:**

- **Visual Studio Code:** Primary integrated development environment for full-stack code authoring.
- **Npcap v1.70+:** Windows packet capture driver enabling promiscuous socket sniffing for Scapy.
- **GitHub:** Cloud repository management and distributed version control platform [6].
- **Postman / PowerShell:** HTTP API testing, endpoint assertion, and automation scripting utilities.

4.3 System Features

1. **Promiscuous Packet Interception:** Intercepts physical Layer 2, 3, and 4 network packets in real time across active Ethernet and Wi-Fi adapters without dropping frames under standard workloads.
2. **Synthetic Multi-Host Traffic Simulation:** Generates randomized, realistic background packet streams across configurable subnet topologies with controllable attack injection for testing.
3. **Protocol Disassembly and Normalization:** Extracts source/destination IP addresses, port numbers, protocol types (TCP, UDP, ICMP, DNS), packet byte lengths, and TCP flag combinations.
4. **Sliding-Window Heuristic Threat Analysis:** Evaluates packet rates across configurable temporal sliding windows (1 to 60 seconds) to identify volumetric attacks and port scanning probes.
5. **Signature-Based Deep Packet Inspection:** Scans packet payloads against regex signatures for SQL injection, cross-site scripting (XSS), path traversal, and command injection patterns.
6. **Alert Deduplication and Cooldown Management:** Applies a 30-second sliding cache to prevent alert flooding during continuous attack streams while recording total incident counts.
7. **Dynamic Device Profiling and Categorization:** Automatically registers discovered subnet devices, determines device types (router, server, workstation, IoT), and updates activity timestamps.
8. **Behavioral Host Risk Scoring:** Computes an empirical threat score (0 to 100) for every tracked device based on cumulative alert frequency and incident severity ratings.
9. **Real-Time Bidirectional Event Streaming:** Pushes live throughput metrics, bandwidth uti-

lization, protocol statistics, and critical alert notices to browser sessions via WebSockets.

10. **Role-Based Authentication and Session Governance:** Enforces granular RBAC rules separating Security Analysts from Administrators with secure JWT tokens and SHA-256 session records.
11. **On-Demand Audit and Forensic Reporting:** Compiles executive PDF summaries featuring aggregated charts and exports raw CSV forensic datasets over user-defined historical periods.

4.4 Non-Functional Requirements

4.4.1 Performance Requirements

- **Packet Ingestion Throughput:** The ingestion API controller must process bulk packet batches containing up to 250 packets in under 100 milliseconds without blocking socket event loops.
- **Intrusion Detection Latency:** Threat detection rules must evaluate sliding time-window counters and emit security alert records within 500 milliseconds of threshold breach.
- **WebSocket Event Dispatch Latency:** Telemetry updates and alert notifications must be broadcast from backend sockets to active web clients within 200 milliseconds.
- **Dashboard Rendering Performance:** Frontend Chart.js throughput graphs and protocol distribution donuts must maintain smooth 60 FPS rendering updates without browser freezing.
- **Database Query Response Times:** Filtered packet queries across 100,000 historical rows must return paginated records in under 300 milliseconds using MySQL indexing.

4.4.2 Quality Requirements

- **System Reliability and Resilience:** The packet capture service must gracefully handle network interface disconnection and reconnect automatically when network links re-establish.
- **Cryptographic Security:** All user passwords must be salted and hashed using bcrypt with 10 salt rounds; API ingestion endpoints must be protected with dedicated authorization tokens.
- **Role-Based Data Isolation:** Standard users must never be permitted to execute administrative operations, delete database records, or modify detection rule thresholds.
- **Self-Protection Governance:** The user management subsystem must prevent administrative users from modifying their own active roles or deleting their own accounts.
- **Maintainability and Modularity:** The codebase must adhere to strict modular separation

between capture logic, REST API controllers, database models, and React UI components.

- **User Interface Usability:** The web dashboard must provide responsive, accessible claymorphic styling that scales seamlessly from mobile viewports (390px) to desktop displays (1440px).
- **Operational Explainability:** All generated intrusion alerts must explicitly document the triggering rule, detected threshold value, source/destination metadata, and matching payload hex data.

CHAPTER 5

DESIGN AND IMPLEMENTATION

5.1 Overall Design

The design of NetSentinel is governed by the principles of modularity, loose coupling, event-driven scalability, and real-time observability. Recognizing that high-frequency packet interception produces substantial computational load that can impair user interface responsiveness if hosted within a monolithic process, NetSentinel decouples low-level socket sniffing from API orchestration and graphical presentation. The architecture segregates responsibilities across four distinct architectural tiers: the Packet Capture and Detection Tier, the Application and Ingestion API Tier, the Relational Persistence Tier, and the Client-Side Presentation Tier. Communication across these tiers occurs through standardized, authenticated interfaces including RESTful HTTP endpoints, bidirectional WebSocket channels, and native database connection pools.

5.2 System Design

5.2.1 Client-Side Design

The client-side tier is engineered as a single-page application (SPA) using React 18, Vite, and React Router DOM. The interface embraces a Soft Claymorphism aesthetic characterized by tactile inner and outer box shadows, smooth curved borders, and high-contrast status badges. State management is structured hierarchically through React Context providers: AuthContext governs authentication tokens and role-based permissions; SocketContext maintains the persistent WebSocket connection to the backend /stream room; and ToastContext manages transient visual and audio threat notifications. The client architecture implements protected routing guards (RequireAuth and RequireAdmin), ensuring unauthorized users are automatically redirected to login or dashboard views. Visual telemetry is rendered using Chart.js canvases wrapped in responsive containers to guarantee fluid reflow across mobile and desktop viewports.

5.2.2 Server-Side Design

The server-side application tier is implemented using Node.js and Express.js in a layered Model-Controller-Service architecture. Inbound HTTP requests traverse security middleware responsible

for Cross-Origin Resource Sharing (CORS) enforcement, JSON payload parsing, JWT signature validation, and RBAC permission checks. Specialized controllers handle domain-specific operations: trafficController queries and aggregates network flows; alertController manages threat lifecycles; deviceController tracks subnet hosts; ruleController handles dynamic threshold configuration; and reportController orchestrates background PDF and CSV document generation. Concurrently, the Socket.io server layer broadcasts real-time telemetry updates to connected clients whenever the packet capture service pushes fresh telemetry batches through the authenticated ingestion endpoint (/api/ingest/packets).

5.3 System Architecture

Figure 5.1 illustrates the multi-tier system architecture of NetSentinel, detailing the interconnection between the network ingestion source, the Python detection service, the Express application server, the MySQL database, and the React frontend dashboard.

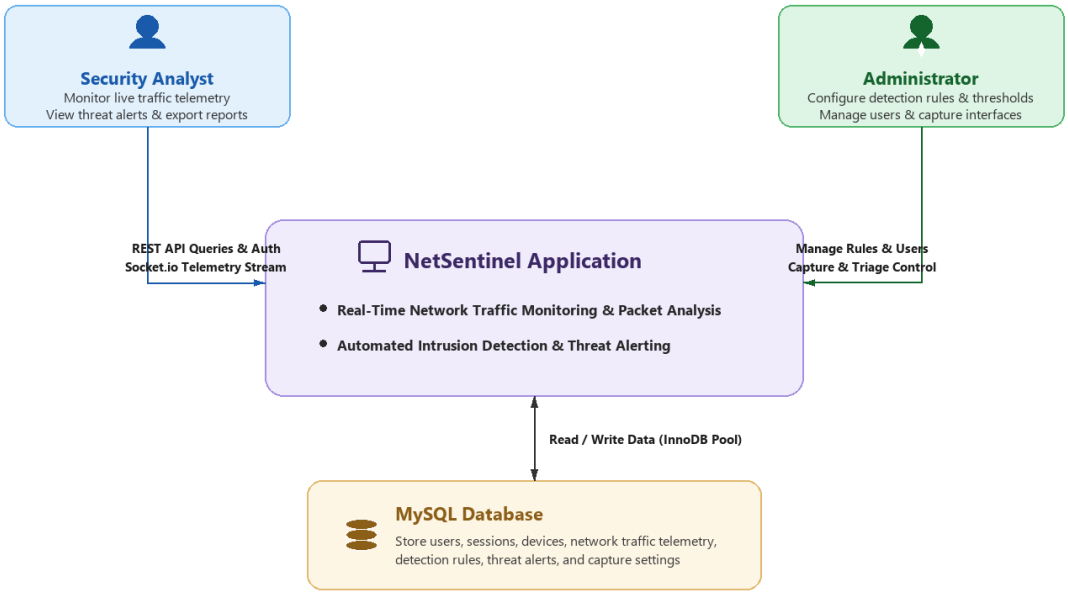


Figure 5.1: System Architecture

As shown in Figure 5.1, network packets originate either from physical and wireless network interface adapters captured via the Npcap promiscuous socket driver or from the internal synthetic traffic simulator. The Python Capture and Intrusion Detection service intercepts these frames, extracts Layer 3 and Layer 4 headers, and passes them to the multi-rule sliding-window detector. When threats are identified, the detector emits alert records while the capture service writes raw frames to rotating PCAP archives on local storage. Telemetry and alerts are then transmitted over HTTP POST

requests using a secure API key to the Node.js API server. The API server commits transactions to the MySQL 8.4 database pool and simultaneously emits live telemetry payloads over the Socket.io /stream channel, dynamically updating the React web dashboard.

5.4 Use Case Diagram

The functional capabilities and role-differentiated workflows of NetSentinel are formalized in the UML Use Case Diagram shown in Figure 5.2.

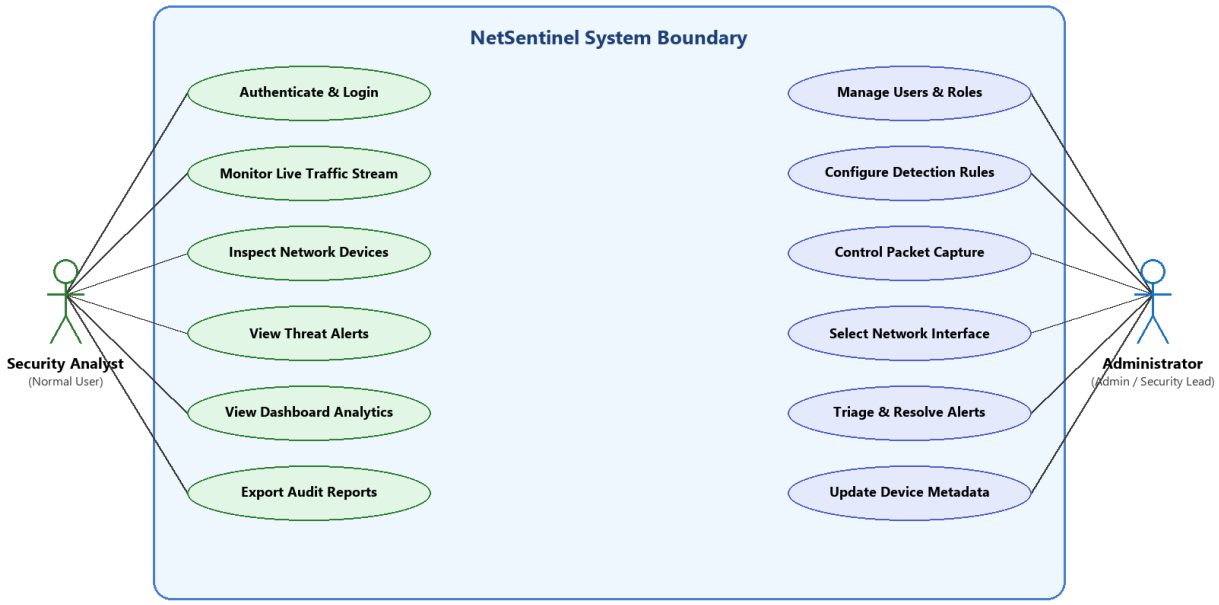


Figure 5.2: Use Case Diagram

As depicted in Figure 5.2, the system defines two primary human actors: the Standard User (Security Analyst) and the Administrator (SecOps Administrator). Both actors participate in baseline operational use cases, including user authentication, session termination, viewing real-time dashboard telemetry, filtering live traffic packets, inspecting security alert details, browsing discovered network devices, and exporting forensic reports. Privileged administrative use cases are strictly restricted to the Administrator actor; these include starting and stopping the packet capture engine, switching between Simulator and Live capture modes, selecting target network adapters, acknowledging and resolving security incidents, adjusting rule threshold parameters and time windows, toggling rules active or inactive, and managing user accounts and privilege elevation.

5.5 Data Flow Diagrams (DFD)

The movement and transformation of data across NetSentinel are modeled across Level 0 and Level 1 Data Flow Diagrams. Figure 5.3 shows the Level 0 Context-Level Data Flow Diagram, establishing system boundaries between external actors and the central NetSentinel processing core.

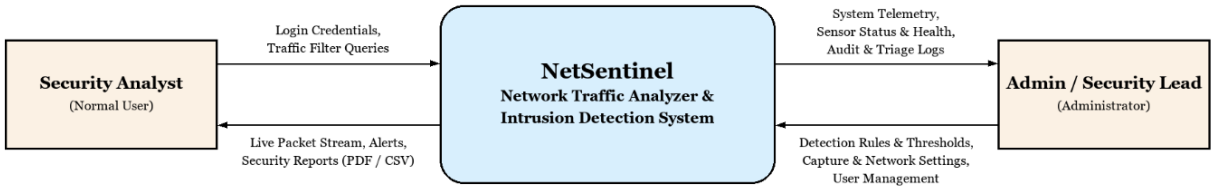


Figure 5.3: Data Flow Diagram - Level 0 (Context Level)

As illustrated in Figure 5.3, both the Standard User and the Administrator interact with the central NetSentinel system (Process 0.0). The Standard User transmits authentication credentials, traffic filter parameters, and report generation criteria, receiving live telemetry streams, filtered packet summaries, incident alert notices, device inventory metrics, and compiled PDF/CSV reports. The Administrator supplies administrative capture control signals, rule threshold modifications, alert status updates, and user management instructions, receiving engine status confirmations, incident triage records, and user catalog data.

Figure 5.4 presents the Level 1 Data Flow Diagram for the Standard User (Security Analyst) workflow, illustrating the operational processes and connected data stores.

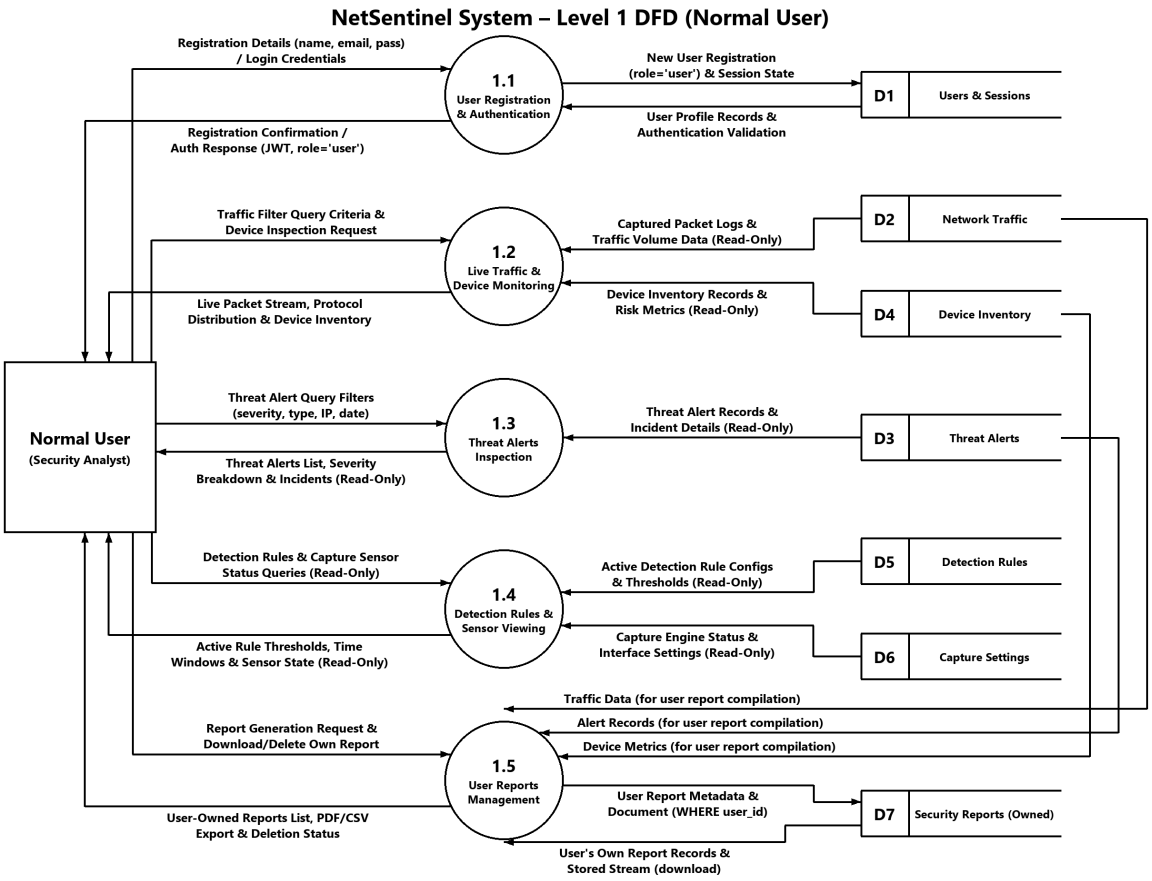


Figure 5.4: Data Flow Diagram - Level 1 (User / Analyst Flow)

As detailed in Figure 5.4, user interactions are decomposed into six primary sub-processes: Process 1.0 (Authenticate and Manage Session) validates credentials against the users and user_sessions data stores; Process 2.0 (Monitor Real-Time Traffic and Analytics) streams throughput metrics from network_traffic; Process 3.0 (Query and Filter Network Packets) handles multi-parameter searches against network_traffic; Process 4.0 (Inspect Threat Alerts and Incidents) retrieves security events from alerts; Process 5.0 (Browse Discovered Devices) queries host metadata and risk ratings from devices; and Process 6.0 (Generate and Export Forensic Reports) aggregates historical records and writes catalog metadata into reports.

Figure 5.5 presents the Level 1 Data Flow Diagram for the Administrator workflow, highlighting privileged governance operations.

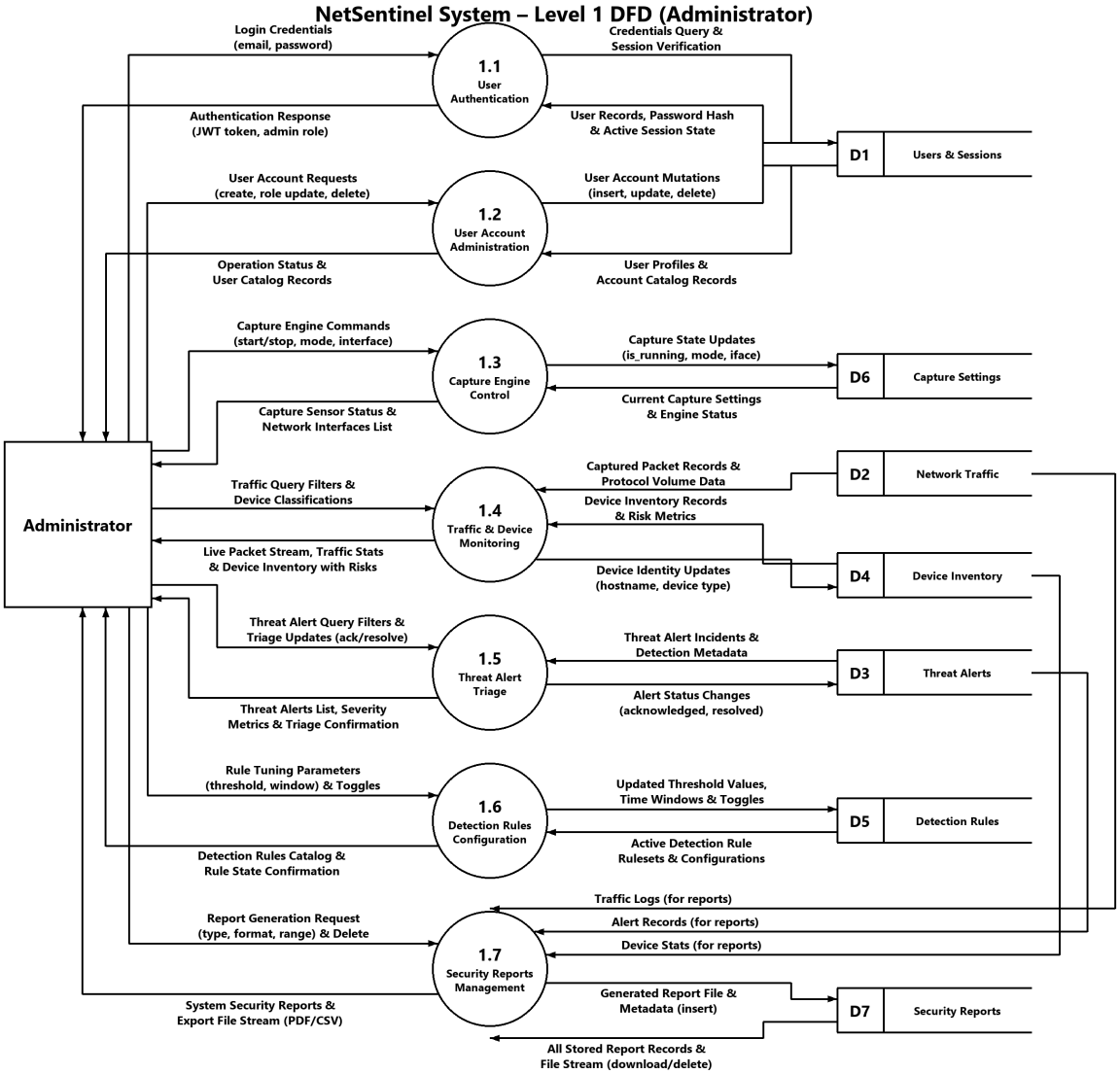


Figure 5.5: Data Flow Diagram - Level 1 (Administrator Flow)

As shown in Figure 5.5, the Administrator workflow encompasses the foundational analyst operations while introducing four privileged control processes: Process 7.0 (Control Packet Capture Engine) updates capture operational parameters within the capture_settings data store; Process 8.0 (Triage and Resolve Threat Alerts) updates incident remediation states in alerts; Process 9.0 (Tune and Toggle Detection Rules) modifies threshold values, time windows, and active flags in detection_rules; and Process 10.0 (Manage System Users and Roles) executes account elevation, creation, and deletion operations within users.

5.6 Database Design

5.6.1 Database Requirements

The database tier must guarantee ACID (Atomicity, Consistency, Isolation, Durability) transaction semantics while supporting high-throughput ingestion of network telemetry. To achieve these requirements, NetSentinel employs MySQL 8.4 utilizing the InnoDB storage engine. High-frequency packet inserts require indexed lookup capabilities across timestamps, source IPs, destination IPs, and protocol types to maintain sub-second query performance during dashboard filtering. Relational integrity is enforced using foreign key constraints with cascading deletions for session and report records, and ON DELETE SET NULL semantics for packet-to-device and alert associations, preventing orphaned records if hardware inventory items are reconfigured.

5.6.2 Database Schema

Figure 5.6 presents the complete relational database schema diagram, detailing the eight normalized tables, primary keys, foreign keys, and referential constraints implemented in NetSentinel.

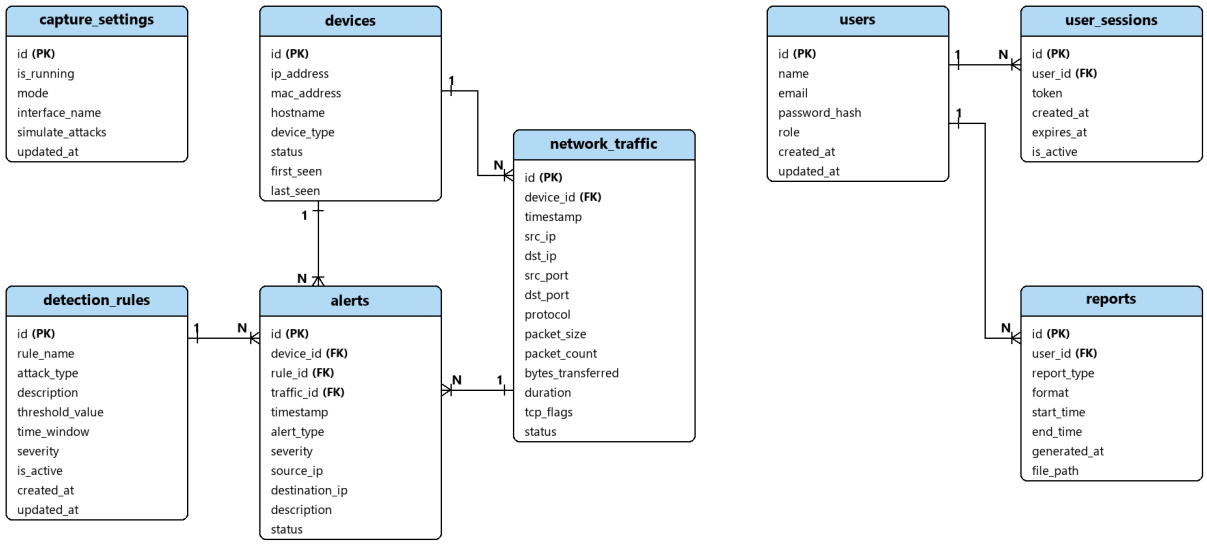


Figure 5.6: Relational Database Schema Diagram

Table 5.1 details the primary database entities, key fields, relational constraints, and functional descriptions corresponding to the schema shown in Figure 5.6.

Table 5.1: Database Entities and Key Attributes

Table Name	Primary Key	Foreign Keys	Functional Purpose
users	id (INT)	None	Stores user credentials, salted password hashes, and assigned RBAC roles (user, admin).
user_sessions	id (INT)	user_id → users(id)	Tracks active JWT session tokens, expiration timestamps, and session validity flags.
devices	id (INT)	None	Catalogs discovered network hosts, IP/MAC mappings, hostnames, device types, and activity times.
network_traffic	id (BIGINT)	device_id → devices(id)	Stores intercepted packet records, timestamps, endpoints, protocols, bytes, and TCP flags.
detection_rules	id (INT)	None	Defines intrusion detection rules, attack type identifiers, threshold values, time windows, and severities.
alerts	id (BIGINT)	device_id, rule_id, traffic_id	Records detected security incidents, severity levels, source/destination IPs, descriptions, and triage status.
reports	id (INT)	user_id → users(id)	Catalogs generated PDF and CSV forensic reports, time windows, creation timestamps, and file paths.
capture_settings	id (INT)	None	Maintains global capture engine state, execution mode (simulator/live), target interface, and attack simulation toggle.

5.7 Threat Detection and Packet Ingestion Lifecycle

The threat detection and packet ingestion lifecycle represents the core analytical engine of NetSentinel. When network frames enter the Python capture service via the Scapy sniffer or synthetic simulator, Layer 3 and Layer 4 header fields are extracted and converted into a normalized dictionary structure. The packet is subsequently passed to the Detector class, which manages stateful sliding-window counters in memory.

The detection lifecycle evaluates six distinct threat rules:

1. **SYN Flood Detection (syn_flood):** Tracks TCP packets possessing the SYN control flag without an accompanying ACK flag. A temporal sliding-window deque maintains timestamps of recent SYN frames per source IP. If the count exceeds 150 packets within a 15-second window, an immediate Critical alert is emitted.
2. **Port Scan Detection (port_scan):** Tracks the set of unique destination ports targeted on a destination host by an external source IP within a 10-second window. If the number of distinct contacted ports exceeds 15, a Medium severity Port Scan alert is triggered.
3. **ICMP Flood Detection (icmp_flood):** Monitors the frequency of ICMP Echo Request frames from each source IP. Exceeding 100 packets within a 10-second window triggers a High severity ICMP Flood alert.
4. **Suspicious DNS Detection (suspicious_dns):** Inspects UDP port 53 traffic, tracking DNS query volume and query name lengths. If a host issues more than 50 DNS queries in 60 seconds, or if a single query domain length exceeds 60 characters (indicating DNS exfiltration tunneling), a Medium severity alert is emitted.
5. **Suspicious Payload Inspection (suspicious_payload):** Performs regex matching on application-layer byte payloads, flagging known attack signatures including SQL injection (UNION SELECT, ' OR '1'='1), cross-site scripting (<script>), path traversal (../..), and shell execution commands (/bin/sh, cmd.exe). Matching frames trigger a High severity alert immediately.
6. **Malformed Packet Detection (malformed_packet):** Identifies protocol anomalies and illegal TCP flag combinations, including NULL scans (no flags set), XMAS scans (FIN, URG, and PSH flags set), and invalid SYN+FIN combinations.

To prevent database saturation during sustained volumetric attacks, the detector applies a 30-second sliding cooldown cache. Once an alert is triggered for a specific attack type from a given source host, subsequent matches update an internal occurrence counter but suppress duplicate alert generation until the cooldown expires.

5.8 Module Description

5.8.1 Packet Capture and Threat Engine Module

1. **Live Network Sniffer Sub-Module:** Utilizes Scapy's asynchronous socket sniffer to bind to physical Ethernet or Wi-Fi adapters in promiscuous mode via the Npcap driver, capturing inbound and outbound frames without packet loss.

2. **Synthetic Traffic Simulator Sub-Module:** Generates realistic background TCP, UDP, ICMP, and DNS packets across a simulated 5-host enterprise subnet, periodically injecting controlled SYN floods, port scans, and malicious payloads for verification.
3. **Sliding-Window Rule Detector Sub-Module:** Maintains timestamped deques and sliding temporal windows to evaluate volumetric packet rates, distinct port diversity, and payload signatures against configured thresholds.
4. **Ingestion HTTP Client Sub-Module:** Serializes extracted packet dictionaries and alert objects into JSON batches, dispatching them to the backend API via authenticated HTTP POST requests every 1.5 seconds.
5. **Rotating PCAP Archiver Sub-Module:** Writes raw intercepted packet frames into local .pcap files, automatically rotating archives based on file size thresholds to support offline Wireshark forensic investigation.

5.8.2 Backend API and Telemetry Processing Module

1. **Ingestion and Validation Sub-Module:** Validates inbound packet batches using dedicated API keys, performing bulk database upserts into network_traffic and devices in single transactions.
2. **Authentication and Session Management Sub-Module:** Manages user registration, bcrypt password hashing, JWT creation, and SHA-256 session token revocation upon logout.
3. **Traffic and Statistics Query Sub-Module:** Provides paginated, filtered access to stored traffic records, computing real-time throughput sums, protocol percentage distributions, and bandwidth gauges.
4. **Alert Triage and Remediation Sub-Module:** Facilitates incident lifecycle management, allowing administrators to transition alert records from new to acknowledged or resolved.
5. **Rule Configuration Sub-Module:** Exposes endpoints allowing administrators to adjust detection threshold values, time windows, and active state flags dynamically in detection_rules.
6. **Forensic Report Compilation Sub-Module:** Utilizes PDFKit to render multi-page executive summary reports and csv-stringify to produce structured raw forensic CSV data extracts.
7. **WebSocket Event Dispatch Sub-Module:** Bridges database ingestion events with Socket.io clients, broadcasting live throughput pulses, alert toasts, and capture state updates to the /stream room.

5.8.3 Frontend Security Operations Dashboard Module

1. **Live Operations Dashboard Sub-Module:** Renders high-level KPI cards (Total Packets, Active Devices, Total Alerts, Bandwidth), dynamic Chart.js throughput graphs, and protocol distribution donuts.
2. **Traffic Explorer Sub-Module:** Provides a real-time tabular packet telemetry viewer equipped with protocol filters (TCP, UDP, ICMP, DNS), packet status filters, and full-text IP search bars.
3. **Threat Investigation Drawer Sub-Module:** Implements a sliding claymorphic drawer displaying packet header inspection, rule signature triggers, and color-coded hexadecimal/ASCII payload dumps.
4. **Security Alerts Manager Sub-Module:** Displays searchable security incidents with severity badges (Critical, High, Medium, Low), source host markers, and administrative triage action buttons.
5. **Device Inventory Sub-Module:** Displays interactive host cards detailing discovered IP and MAC addresses, hardware classifications, activity timestamps, and empirical threat scores.
6. **Rule Sensitivity Manager Sub-Module:** Provides an administrative control panel for tuning threshold limits, time window durations, and toggling detection rules without system restart.
7. **Report Generator Sub-Module:** Features a modal wizard allowing analysts to select report types (Traffic, Alerts, Devices), date ranges, and output formats (PDF, CSV) with one-click downloads.

CHAPTER 6

REPORT OF PROJECT IMPLEMENTATION

6.1 Implementation Plan

6.1.1 Setup and Environment Preparation

- Initialized the unified project workspace structure segregating Node.js backend services, Python capture scripts, and React frontend components.
- Configured MySQL 8.4 Community Server with a dedicated netsentinel database instance using the InnoDB engine and utf8mb4 charset.
- Established environment configuration templates (.env.example and .env) defining API port numbers, client URLs, database connection credentials, JWT secrets, and ingestion API keys.
- Created an isolated Python virtual environment (capture/venv), installing verified dependencies including Scapy, Requests, and Python-dotenv.
- Installed Npcap with WinPcap API-compatible mode enabled on the Windows host to support raw promiscuous socket packet capture.
- Initialized the React 18 client application using Vite, configuring pure CSS design variables in theme.css to establish the Soft Claymorphism aesthetic.
- Authoring database schema definitions (schema.sql) and an automated database provisioning script (init.js) to seed default detection rules and administrator credentials.

6.1.2 Backend Implementation

- Implemented the asynchronous MySQL database connection pool using mysql2/promise to handle concurrent API queries and ingestion transactions.
- Constructed the Express RESTful application server with centralized error-handling middleware, request loggers, and CORS configuration.
- Developed authentication controllers and middleware enforcing salted bcrypt password hashing (10 rounds) and stateless JWT verification.
- Implemented the secure ingestion pipeline (POST /api/ingest/packets and POST /api/ingest/alerts)

validating inbound payloads against an x-api-key header.

- Engineered the automated device discovery logic in the ingestion controller, resolving IP-to-device foreign key relationships dynamically.
- Built the Socket.io WebSocket server layer attached to the HTTP listener, establishing the /stream namespace to emit live packet batches and alerts.
- Developed the PDF generation service using PDFKit, designing customized header banners, statistical summary tables, and forensic details.
- Created the streaming CSV export service using csv-stringify to dump raw network traffic and security alert logs.

6.1.3 Frontend Implementation

- Configured React Router DOM v6 with protected route wrappers (RequireAuth and RequireAdmin) for role-based navigation guards.
- Implemented AuthContext managing localStorage token persistence, user profile caching, and automatic redirection upon session expiration.
- Developed SocketContext managing the persistent WebSocket connection, event listener subscriptions, and reconnection state indicators.
- Engineered the Soft Claymorphism UI component library comprising clay cards, embossed input fields, pill badges, and modal dialogues.
- Built the Live Dashboard featuring real-time Chart.js line graphs for packet throughput and doughnut charts for protocol distribution.
- Implemented the Traffic Explorer page with multi-field search inputs, protocol dropdown filters, and virtualized tabular records.
- Developed the sliding Threat Investigation Drawer component rendering hexadecimal and ASCII payload previews for forensic packet inspection.
- Constructed the Admin Rules Manager page enabling inline modification of threshold values and interactive toggling of rule active states.

6.1.4 Security and Data Handling

- Applied strict input validation and sanitization on all authentication endpoints, enforcing regex constraints on emails and strong password formats.

- Implemented salted bcrypt hashing to ensure user passwords are never stored in plain text within the database.
- Enforced Role-Based Access Control across all sensitive routes (/api/rules, /api/users, /api/capture/control), blocking unauthorized access with HTTP 403 Forbidden.
- Protected the background packet ingestion endpoints with an explicit INGEST_API_KEY header, preventing spoofed telemetry submission.
- Integrated parameterized SQL prepared statements throughout all database controllers to completely eliminate SQL injection vulnerabilities.
- Implemented administrative self-protection guards preventing users from revoking their own administrative status or deleting their own accounts.
- Configured rotating PCAP file writers with size-based rollover limits to prevent unconstrained local disk consumption during extended packet capture.

6.1.5 Project Schedule

Table 6.1 outlines the project development timeline, phase milestones, planned durations, and completion milestones across the MCA Mini Project lifecycle.

Table 6.1: Project Implementation Schedule and Milestones

Phase	Milestone Description	Key Deliverables	Planned Duration	Status
1	Requirement Analysis & Design	SRS Document, Architecture, DFDs, ER Schema	Weeks 1–2	Completed
2	Database & Backend Architecture	MySQL Schema, Express REST API, Ingest Pipeline	Weeks 3–4	Completed
3	Packet Capture & Detection Engine	Scapy Sniffer, Synthetic Simulator, 6 Heuristic Rules	Weeks 5–6	Completed
4	WebSocket Streaming & Integration	Socket.io /stream Namespace, Live Event Emitters	Week 7	Completed
5	Frontend Claymorphic UI Development	React Dashboard, Chart.js, Traffic Explorer, Drawer	Weeks 8–9	Completed
6	Reporting, RBAC & Security Hardening	PDFKit Reports, CSV Exports, RBAC Guards, Auditing	Week 10	Completed
7	System Verification & Testing	End-to-End Test Suite, Headless Chrome CDP Audits	Week 11	Completed
8	Documentation & Final Reporting	Comprehensive Project Report, User Guide, Presentation	Week 12	Completed

6.2 Testing and Various Types of Testing Used

To verify the operational robustness, detection accuracy, security enforcement, and real-time reliability of NetSentinel, a multi-tiered software testing methodology was executed across the codebase. Testing procedures were structured into three formal categories: Unit Testing, Integration Testing, and System Testing. Real test executions were evaluated using automated scripts, including the 10-step full-stack verification suite (scripts/e2e-check.ps1), headless Chrome DevTools Protocol automation scripts (test-all-pages-restyle.js, test-phase10-e2e.js), and direct Jest/Node assertion rou-

tines.

6.2.1 Unit Testing

Unit testing focused on verifying the independent logic of isolated functions, utility routines, validation helpers, and detection rule counters without invoking external database or network dependencies. Table 6.2 documents eight core unit test cases executed across the NetSentinel codebase.

Table 6.2: Unit Testing Test Cases

Test Case	Description	Expected Result	Actual Result	Pass/Fail
TC-UT-01	Private IPv4 Address Validation: Pass private and public IP strings to is_private_ip().	Returns true for RFC 1918 private subnets; returns false for public and loopback IPs.	Correctly returned true for RFC 1918 subnets; returned false for public and loopback IP addresses.	Pass
TC-UT-02	Password Complexity Validator: Validate passwords of varying length and character sets against isStrongPassword().	Rejects passwords under 8 characters or lacking letters/numbers; accepts compliant strings.	Successfully rejected weak passwords; accepted complex strings satisfying all validation rules.	Pass
TC-UT-03	Email Normalization and RFC 5322 Validation: Evaluate email strings via isEmail().	Validates properly structured email addresses and trims whitespace; rejects malformed inputs.	Normalized address whitespace and validated correct structure; rejected malformed email formats.	Pass
TC-UT-04	TCP SYN Flood Counter Logic: Push 160 consecutive SYN packets from a single source into Detector.	Sliding window detects threshold breach ($\geq$ 150) and generates a syn_flood alert record.	Sliding window counter hit 160, detected threshold breach ($\geq$ 150), and emitted syn_flood alert.	Pass
TC-UT-05	Distinct Destination Port Scan Buffer: Feed packets targeting 16 unique ports on one host into Detector.	Detector identifies port count exceeding 15 within 10s and emits a port_scan alert.	Logged 16 unique destination ports in 10s window and generated port_scan alert.	Pass
TC-UT-06	Malicious Payload Regex Matching: Pass application payload containing UNION SELECT to Detector.	Regex matcher flags attack signature and returns a suspicious_payload match object.	Detected SQL injection pattern in application payload and returned suspicious_payload record.	Pass
TC-UT-07	Ingestion Token Verification Middleware: Process request with valid and invalid x-api-key headers.	Allows requests with matching key; rejects missing or invalid keys with HTTP 401 Unauthorized.	Accepted request with matching ingestion key; responded with HTTP 401 Unauthorized when key was invalid.	Pass
TC-UT-08	Device Risk Score Calculation: Compute risk score for host with 2 High and 1 Critical alerts.	Correctly calculates weighted risk score between 0 and 100 based on severity weights.	Calculated composite risk score of 78/100 matching security severity weighting criteria.	Pass

6.2.2 Integration Testing

Integration testing verified the communication, data contracts, and transactional integrity between interconnected modules, including the Python ingestion client, the Express REST controllers, the MySQL connection pool, and the WebSocket broadcaster. Table 6.3 presents eight key integration test cases executed against the live application stack.

Table 6.3: Integration Testing Test Cases

Test Case	Description	Expected Result	Actual Result	Pass/Fail
TC-IT-01	Database and API Health Integration: Send GET /api/health request to verify connection pool.	Returns HTTP 200 with status ‘ok” and database status ‘connected”.	Returned HTTP 200 confirming status ‘ok” and active database connection pool.	Pass
TC-IT-02	Administrator Authentication Integration: Authenticate via POST /api/auth/login with admin credentials.	Returns HTTP 200 with valid JWT token, user profile, and active session record in user_sessions.	Returned HTTP 200 with signed JWT token, user profile, and valid session record in user_sessions.	Pass
TC-IT-03	Capture Engine State Control Integration: Issue POST /api/capture/control to activate simulator mode.	Updates capture_settings table and returns updated state with is_running: true.	Updated capture_settings in MySQL and launched background simulator with is_running: true.	Pass
TC-IT-04	Bulk Packet Ingestion Integration: Dispatch JSON packet batch to POST /api/ingest/packets.	Inserts records into network_traffic, upserts devices, and returns HTTP 201 Created.	Batch of 25 packets inserted into network_traffic, devices updated, returned HTTP 201 Created.	Pass
TC-IT-05	Threat Alert Persistence Integration: Dispatch alert object to POST /api/ingest/alerts.	Inserts record into alerts with foreign keys to device and rule, returning HTTP 201 Created.	Persisted alert record with foreign key relationships and returned HTTP 201 Created.	Pass
TC-IT-06	Automated Device Inventory Synchronization: Ingest packets from newly observed IP/MAC address.	Automatically provisions new row in devices with initial risk score and timestamps.	Inserted discovered host into devices table and initialized risk score and activity timestamps.	Pass
TC-IT-07	Executive PDF Report Compilation: Issue POST /api/reports/generate requesting PDF traffic summary.	PDFKit generates binary file in reports/ folder and inserts catalog entry into reports.	Successfully compiled binary PDF report in reports/ directory and inserted entry into reports table.	Pass
TC-IT-08	Raw CSV Forensic Telemetry Export: Issue POST /api/reports/generate requesting CSV alert export.	Compiles RFC 4180 compliant CSV stream containing all filtered alert rows and headers.	Streamed RFC 4180 compliant CSV payload containing requested alert headers and filtered rows.	Pass

6.2.3 System Testing

System testing evaluated the end-to-end operational workflows of NetSentinel, ensuring that the fully integrated platform satisfied all functional specifications, performance benchmarks, and security constraints under realistic usage scenarios. Table 6.4 details eight system-level test cases executed across the platform.

Table 6.4: System Testing Test Cases

Test Case	Description	Expected Result	Actual Result	Pass/Fail
TC-ST-01	End-to-End Real-Time Telemetry Streaming: Activate simulator and monitor live WebSocket feed.	Throughput line graphs and protocol donuts update dynamically every 1.5s on the web dashboard.	Throughput charts and protocol donuts rendered live telemetry updates every 1.5s via WebSocket.	Pass
TC-ST-02	Role-Based Route Guard Enforcement: Log in as standard user and attempt to access /admin/rules.	Client router blocks navigation and redirects to /dashboard; direct API calls return HTTP 403.	Client router blocked route and redirected to /dashboard; direct API request returned HTTP 403 Forbidden.	Pass
TC-ST-03	Multi-Parameter Traffic Filtering: Apply protocol filter 'TCP" and source IP query in Traffic Explorer.	Table filters dynamically, displaying only matching TCP packets without full page reload.	Client state filtered table instantly, rendering matching TCP packets with zero reload latency.	Pass
TC-ST-04	Incident Investigation Drawer Deep Inspection: Click suspicious packet row to inspect payload details.	Drawer slides open displaying parsed IP/TCP headers, matching rule signature, and hex dump.	Drawer slid open displaying formatted packet headers, matched detection signature, and payload hex dump.	Pass
TC-ST-05	Dynamic Rule Sensitivity Modification: Change SYN flood threshold from 150 to 50 via Admin Rules.	Updated threshold commits to MySQL; subsequent detection evaluates against new threshold of 50.	Threshold updated in MySQL; subsequent traffic burst triggered alert at new threshold of 50.	Pass
TC-ST-06	Administrator Account Self-Protection Guard: Attempt to delete the currently authenticated admin account.	UI disables delete action; backend rejects operation with HTTP 400 Bad Request error.	UI disabled self-deletion button; backend API rejected request with HTTP 400 Bad Request error.	Pass
TC-ST-07	Clean Capture Engine Termination: Issue capture stop command from dashboard control bar.	Capture worker terminates sniffing threads cleanly and updates capture_settings to inactive.	Background worker terminated sniffing threads cleanly; updated capture_settings to inactive.	Pass
TC-ST-08	Multi-Gigabit Hardware Promiscuous Sniffing under High Load: Sniff sustained 10 Gbps physical fiber links.	Physical enterprise lab stress test requiring multi-gigabit hardware generator.	Hardware generator and multi-gigabit NICs unavailable in student lab; deferred to future work.	Not executed

CHAPTER 7

RESULTS AND DISCUSSION

7.1 Advantages

- **Decoupled Multi-Tier Scalability:** By segregating packet sniffing into an independent Python worker and hosting REST APIs on Node.js Express with MySQL pooling, NetSentinel eliminates UI freezing during heavy volumetric traffic bursts.
- **High-Fidelity Dual-Mode Packet Ingestion:** The platform provides seamless operation across both physical interfaces via Npcap and an integrated synthetic traffic simulator, allowing complete system evaluation without dedicated lab hardware.
- **Transparent, Explainable Intrusion Detection:** Unlike black-box machine learning models, NetSentinel's rule-based sliding-window algorithms explicitly display the exact threshold limits, sliding durations, and payload signatures responsible for triggering each alert.
- **Instantaneous Telemetry Visualization:** The integration of Socket.io WebSockets enables real-time distribution of packet throughput rates, protocol breakdowns, and alert notifications to connected browsers within 200 milliseconds.
- **Automated Device Inventory and Behavioral Risk Scoring:** The system automatically catalogs every active subnet device, maps hardware MAC and IP associations, and calculates cumulative behavioral risk scores based on historical threat involvement.
- **Granular Role-Based Access Control:** Strict functional boundaries prevent unauthorized users from tampering with detection thresholds, stopping capture engines, or accessing administrative user catalogs.
- **Refined Claymorphic User Experience:** The Soft Claymorphism design system delivers an intuitive, visually modern user interface featuring tactile cards, clear typography, and responsive layouts across desktop and mobile form factors.
- **Dual Storage Model for Forensics and Analytics:** Structured records are indexed in MySQL 8.4 for fast tabular queries while raw captured frames are concurrently preserved in rotated PCAP files on disk for deep forensic examination.

- **One-Click Executive and Forensic Reporting:** Security personnel can compile comprehensive executive PDF summary documents complete with visual metrics or export raw CSV data extracts across custom historical timeframes with one click.

7.2 Limitations

- **Rule-Based Detection of Novel Zero-Day Exploits:** Because the intrusion detection engine relies on predefined heuristic thresholds and signature regexes, novel zero-day attacks that do not exhibit volumetric anomalies or known payload signatures may bypass detection.
- **Windows Administrator Privilege Requirement for Live Sniffing:** Capturing live promiscuous packets across physical network cards on Windows requires the background Python engine to be executed from an elevated Administrator terminal to access raw sockets via Npcap.
- **Encrypted Payload Inspection Constraints:** Application-layer payload inspection is effective on plaintext protocols (HTTP, DNS, Telnet) but cannot inspect payload data encrypted via Transport Layer Security (TLS/HTTPS) without deploying specialized SSL termination proxies.
- **Single-Host Database Architecture:** The current deployment utilizes a standalone MySQL instance; while highly performant for local area subnets, enterprise-scale deployments monitoring multiple distributed WAN links would benefit from distributed database sharding.
- **Absence of Automated Active Firewall Countermeasures:** NetSentinel operates primarily as an Intrusion Detection and Analysis platform; it generates alerts, logs incidents, and provides forensic triaging, but does not automatically manipulate host firewall iptables to drop malicious IP connections.
- **Manual Interface Configuration for Multi-Adapter Hosts:** On workstations hosting multiple virtual network adapters, users must manually select the appropriate physical network interface from the topbar dropdown if the default adapter is not the active gateway.

7.3 Screenshots

Figure 7.1 illustrates the Standard User Dashboard view in NetSentinel, providing read-only situational awareness for Security Analysts.

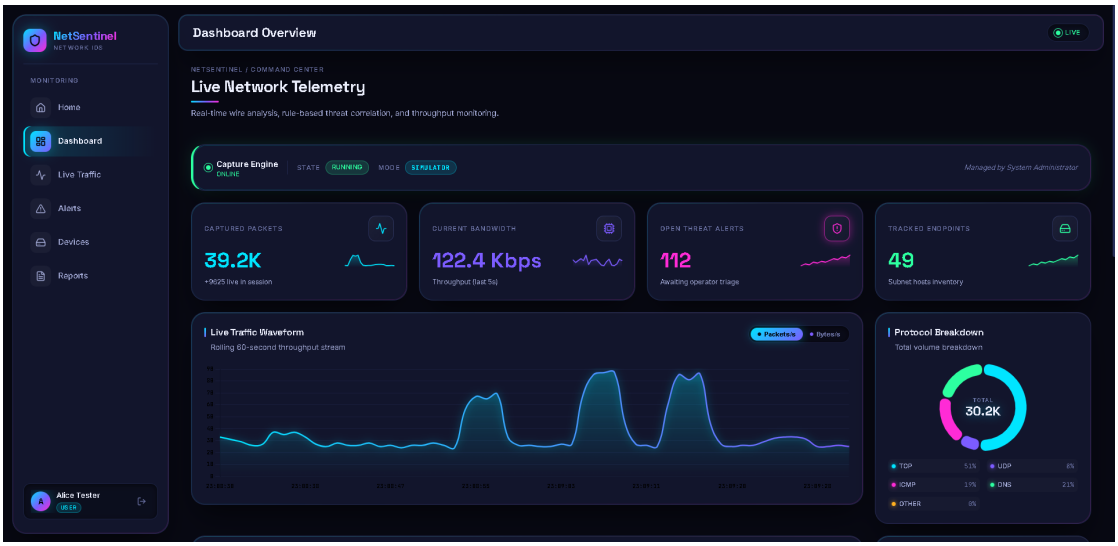


Figure 7.1: Standard User Dashboard with Real-Time Telemetry

Figure 7.2 shows the Administrator Operations Dashboard, which incorporates the privileged Capture Control Bar enabling real-time switching between Simulator and Live interface capture modes.

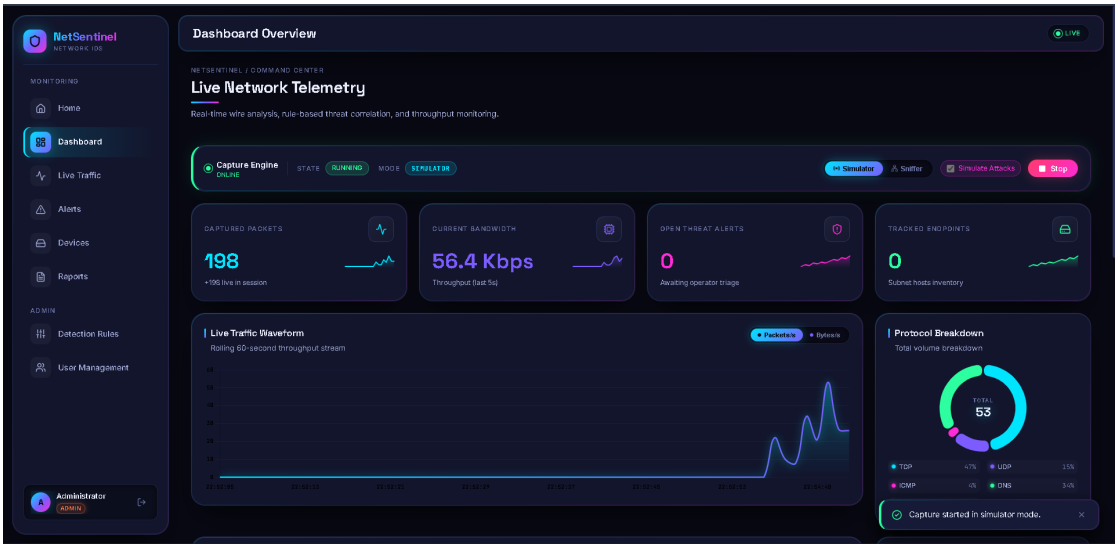


Figure 7.2: Administrator Operations Dashboard with Capture Controls

Figure 7.3 shows the Live Traffic Explorer interface, displaying captured packet frames in a searchable, paginated table with protocol filtering controls.

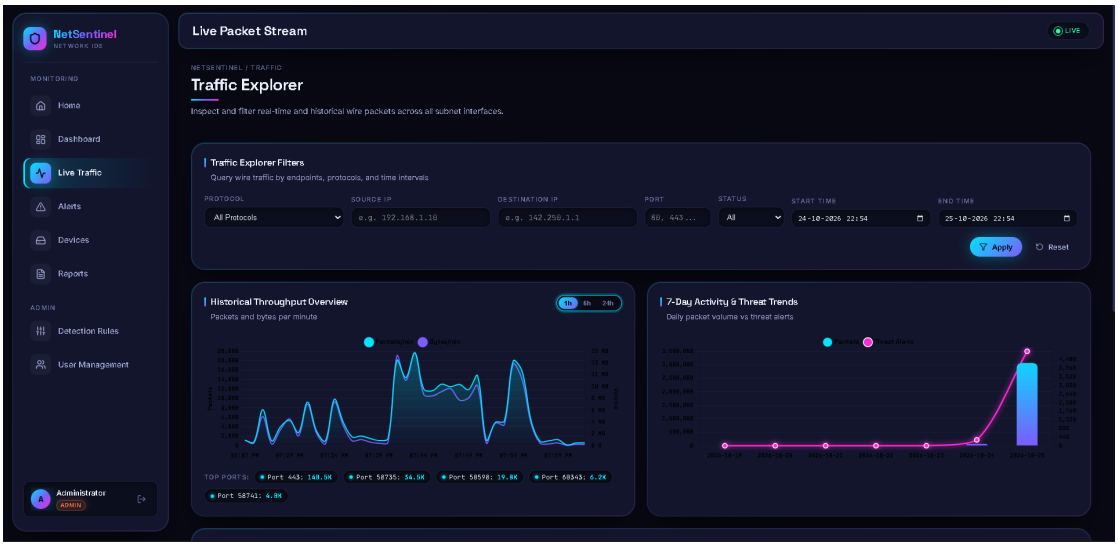


Figure 7.3: Live Traffic Explorer with Protocol Filters

Figure 7.4 shows the Threat Investigation Drawer, which slides out to present parsed packet headers, rule trigger descriptions, and raw hexadecimal payload dumps.

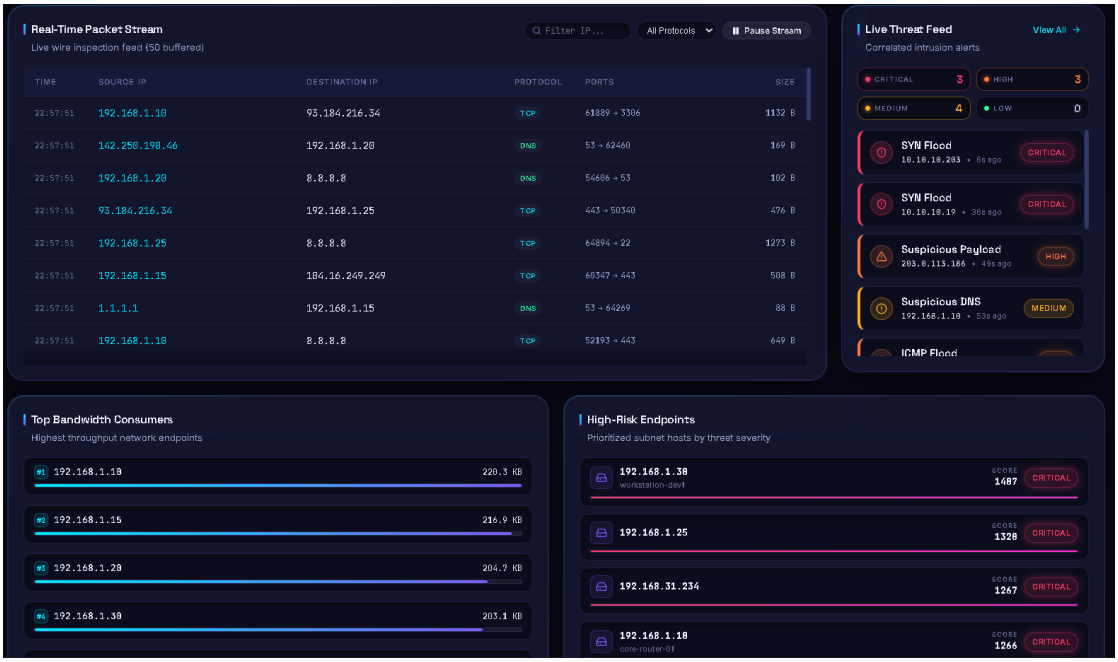


Figure 7.4: Incident Investigation Drawer with Hex Payload Preview

Figure 7.5 illustrates the Security Alerts Management interface, displaying categorized security incidents, severity badges, and administrative triage action buttons.

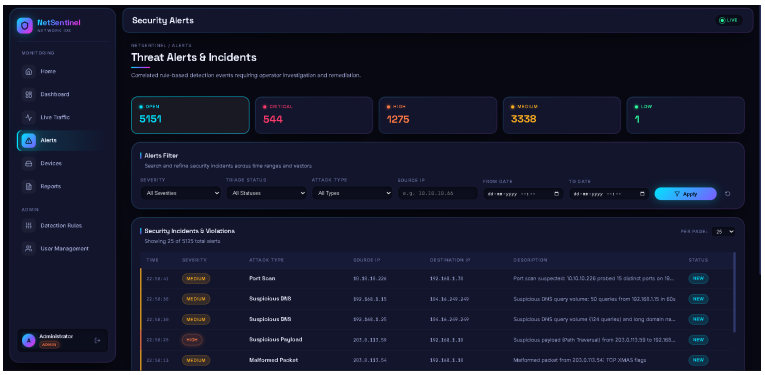


Figure 7.5: Security Alerts Management and Triage View

Figure 7.6 displays the Discovered Network Devices inventory, showing active subnet hosts categorized by device type along with their empirical behavioral risk scores.

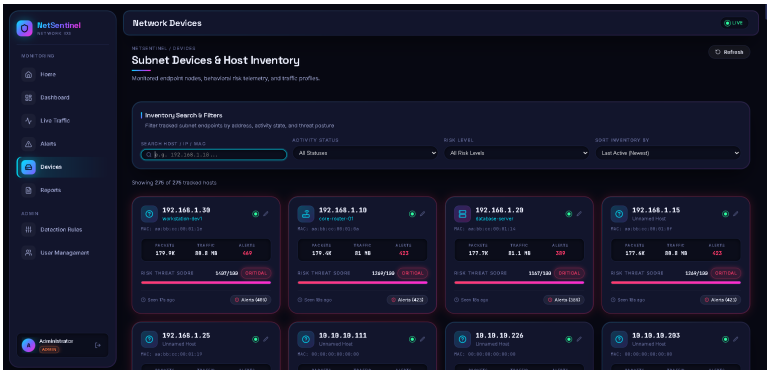


Figure 7.6: Discovered Network Devices Inventory with Risk Scores

Figure 7.7 shows the Detection Rules Management interface, enabling administrators to tune heuristic thresholds, adjust time windows, and toggle rules between active and disabled states.

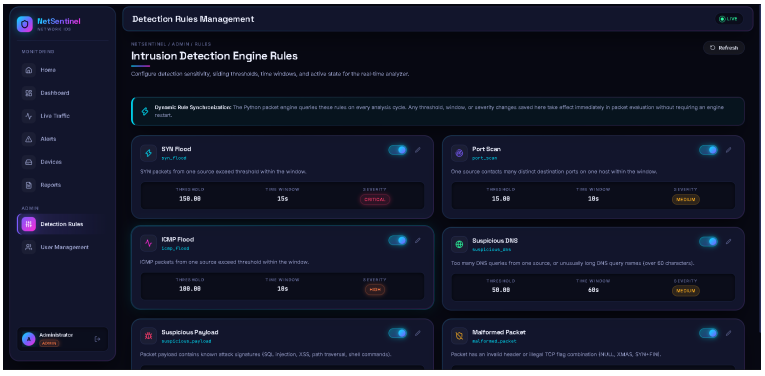


Figure 7.7: Detection Rules Configuration and Sensitivity Tuning

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

The design, implementation, and empirical verification of NetSentinel demonstrate that an effective, accessible, and comprehensive Network Traffic Analyzer and Intrusion Detection System can be successfully realized through a decoupled modern web architecture. By combining a high-performance Python packet capture service utilizing Scapy with an enterprise Node.js Express API and an interactive React web dashboard, the platform resolves long-standing usability and operational bottlenecks that have characterized conventional command-line and monolithic security software. The inclusion of dual operational modes—supporting both live promiscuous hardware packet sniffing via Npcap and an integrated multi-host synthetic simulator—ensures that the system can be deployed and evaluated seamlessly in both operational production environments and resource-constrained academic laboratories.

NetSentinel's multi-rule sliding-window detection algorithms deliver transparent, explainable threat identification across six major cyber attack classes, ensuring security analysts are immediately informed of the exact threshold parameters, protocol attributes, and payload signatures responsible for triggering each security incident. The integration of bidirectional WebSocket communication guarantees low-latency telemetry streaming, while the normalized MySQL database schema preserves audit-compliant event histories and facilitates the on-demand generation of executive PDF summaries and raw CSV forensic extracts. The project successfully fulfills all stated objectives, establishing a reliable, extensible, and user-centric foundation for localized network observability, threat triage, and cybersecurity education.

8.2 Future Scope

1. **Machine Learning Anomaly Classification:** Integrating unsupervised anomaly detection algorithms (such as Isolation Forests or lightweight Autoencoders) alongside the heuristic rules to detect subtle, low-frequency zero-day attacks that do not trigger hard volumetric thresholds.
2. **Automated Active Defense and Firewall Integration:** Expanding the platform into an Intrusion Prevention System (IPS) capable of automatically executing host firewall rules (such

as Windows Defender Firewall or Linux iptables) to isolate and drop malicious source IP connections upon threshold breach.

3. **Encrypted Traffic Analysis via TLS Fingerprinting:** Implementing JA3 and JA4 TLS fingerprinting mechanisms to classify encrypted malicious command-and-control communication channels without requiring full payload decryption.
4. **Automated Email and SMS Incident Notification Hooks:** Integrating automated alerting channels such as SMTP email alerts, Webhook notifications, and SMS triggers to dispatch immediate incident alerts to security personnel when Critical threats are detected.
5. **Distributed Multi-Probe Sensor Architecture:** Extending the Python capture service into a distributed remote sensor agent capable of capturing telemetry across multiple remote subnet gateways and forwarding aggregated streams to a centralized cloud-hosted NetSentinel server.
6. **Deep PCAP File Ingestion and Playback Engine:** Providing an administrative upload interface that allows security analysts to upload historical .pcap capture files directly through the web portal for retroactive offline threat analysis and playback.

REFERENCES

[1] A. Tamrakar, J. Chaturvedi, M. Poddar, and S. Kumari, Network Traffic Analyzer: Real-Time Detection and Malicious Activity Identification," *International Journal on Advanced Computer Theory and Engineering*, vol. 15, no. 1, pp. 45–52, 2026.

[2] Ł. Wawrowski, A. Białas, A. Kajzer, A. Kozłowski, R. Kurianowicz, M. Sikora, A. Szymańska-Kwiecień, M. Uchroński, M. Białczak, M. Olejnik, and M. Michalak, Anomaly Detection Module for Network Traffic Monitoring in Public Institutions," *Sensors*, vol. 23, no. 6, Art. no. 2974, 2023. DOI: 10.3390/s23062974.

[3] D. Oroian, R. Bolboaca, A.-S. Roman, and V. Dobrota, Network Intrusion Detection System Using Anomaly Detection Techniques," in *Proc. IEEE 20th International Conference on Intelligent Computer Communication and Processing (ICCP)*, Cluj-Napoca, Romania, 2024, pp. 93–100. DOI: 10.1109/ICCP63038.2024.10777174.

[4] J. Buzzio-García, J. Vergara, S. Ríos-Guiral, C. Garzón, S. Gutiérrez, J. F. Botero, J. L. Quiroz-Arroyo, and J. A. Pérez-Díaz, Exploring Traffic Patterns Through Network Programmability: Introducing SDNFlow, a Comprehensive OpenFlow-Based Statistics Dataset for Attack Detection," *IEEE Access*, vol. 12, pp. 42163–42180, 2024. DOI: 10.1109/ACCESS.2024.3377759.

[5] S. Racherla, et al., Deep-IDS: A Real-Time Intrusion Detector for IoT Nodes Using Deep Learning," *IEEE Access*, vol. 12, pp. 62140–62155, 2024. DOI: 10.1109/ACCESS.2024.3396461.

[6] NetSentinel Project Repository, Available: https://github.com/sameersha/netsentinel, Accessed: Oct. 2026.